

Connect-8

Four-in-a-Row on an 8x8 Board

ECE Final Project

Team Members:

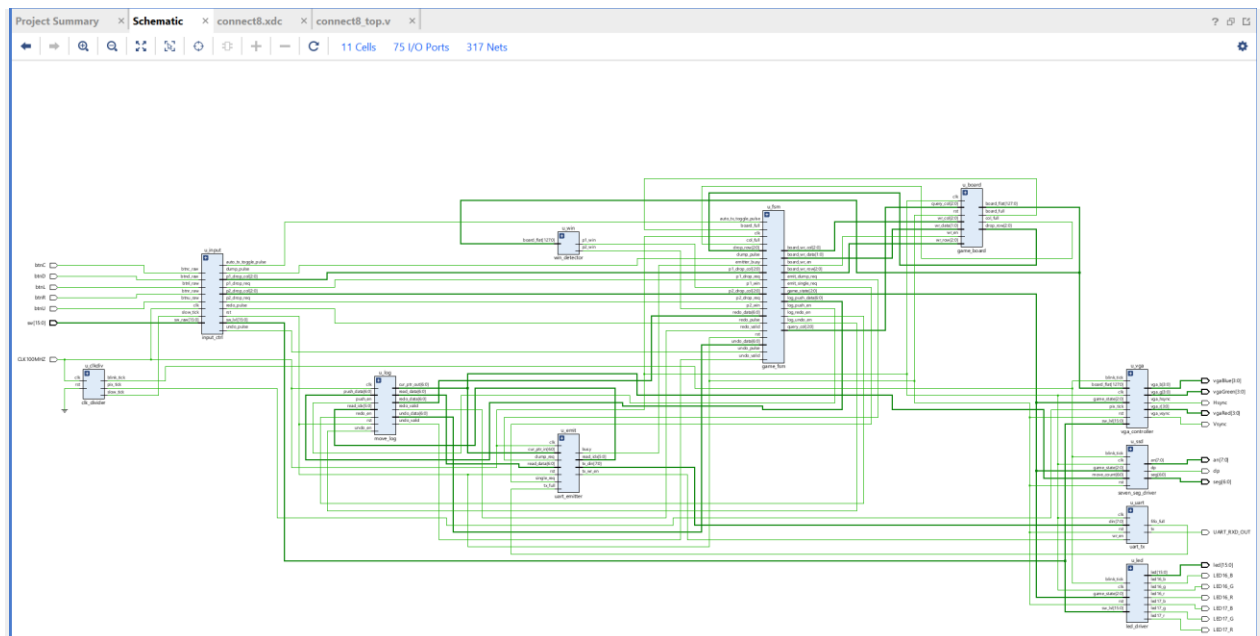
- Justin Ruiz
- Ethan Tungpalan
- Joshua Navarrete
- David Calvillo

1. Description of the System

Connect-8 is our take on Connect Four but played on a bigger 8x8 board. The name comes from the board size, not the win condition, you still need 4 in a row to win. Players take turns dropping pieces into any of the 8 columns, and each piece falls to the lowest open spot. Four in a row in any direction, horizontal, vertical, or diagonal, essentially wins the game.

We implemented the game on a Digilent Nexys A7-100T FPGA using Verilog. The design is made up of 15 modules and a shared constants header file. Everything is clocked off the board's built-in 100 MHz oscillator. The top-level module, connect8_top, connects all the submodules together and maps them to the board's physical pins.

A diagram of the module hierarchy is shown below.



2. Required I/Os

The design uses all six required I/O types on the Nexys A7 board.

Switches (SW0 through SW15)

Player 1 uses SW0–SW7 to pick a column, and Player 2 uses SW8–SW15. Flipping a switch up drops a piece in that column, which then falls to the lowest open spot. Each switch goes through a debouncer and edge detector in `input_ctrl`, so holding a switch up won't keep dropping pieces, you have to flip it down and back up to drop again.

Push Buttons

All 5 buttons on the board are used:

- BTNC: system reset, clears the board and starts a new game
- BTNL: undo, removes the most recent move
- BTNR: redo, restores a move that was undone
- BTNU: manual UART dump, sends the whole game history over UART
- BTND: toggles whether moves automatically print on UART when they happen (default is on)

LEDs

LD0 through LD15 are off by default during normal play. When the game ends, they light up in a one hot chase pattern at a 2Hz frequency regardless of a player win or draw. The RBD LEDs LD16 and LD17 at the top of the board light up red when it's Player 1's turn and blue for Player 2's turn. On a draw they both light white.

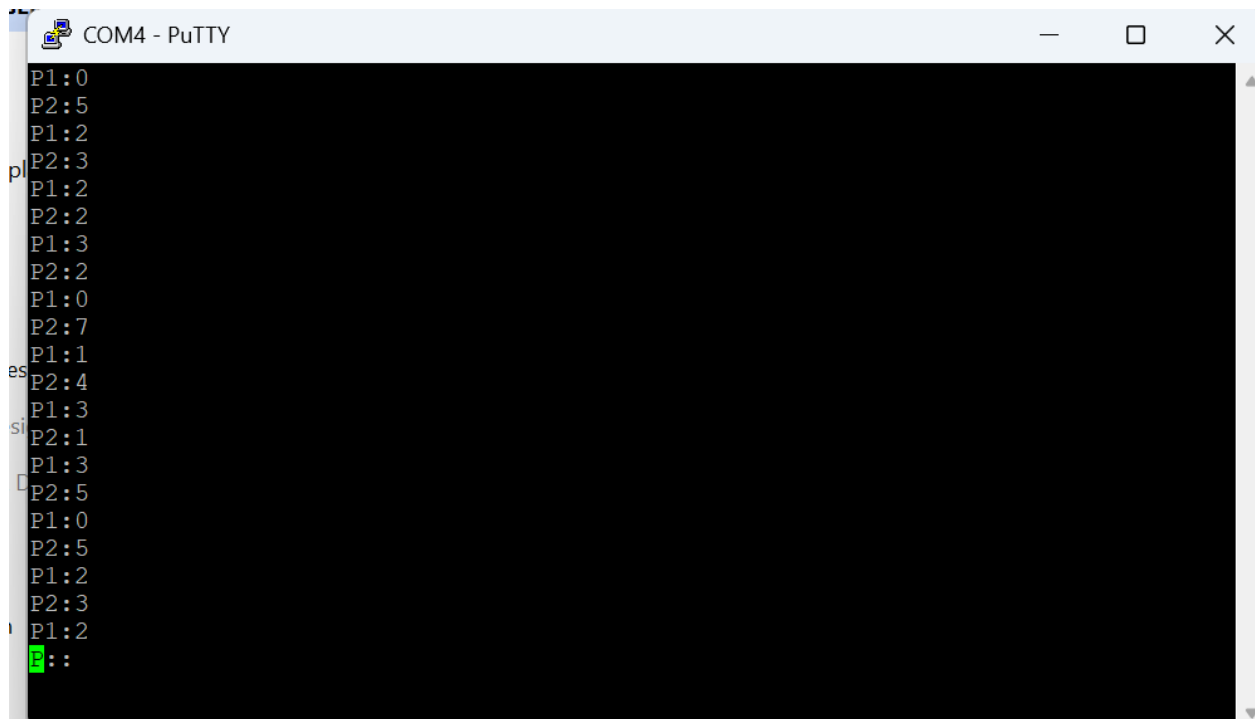
7 Segment Display

We use the display to show the current player and the move count during normal play. The two left digits show "P1" or "P2" depending on whose turn it is. The two right digits show the number of moves played so far in two digit format. When the game ends, "P1" or "P2" blink on the left side, or the whole display shows dashes for a draw.

UART

The FTDI chip on the Nexys A7 appears as a virtual COM port on Windows. Our UART transmitter is set to 9600 baud, 8 data bits, no parity, 1 stop bit. After each move, the FPGA sends a 6 byte message in the format "P1:3\r\n" (or whatever the player and column are). PuTTY or any other serial terminal can be used to view the output.

Screenshot of PuTTY Server on next page



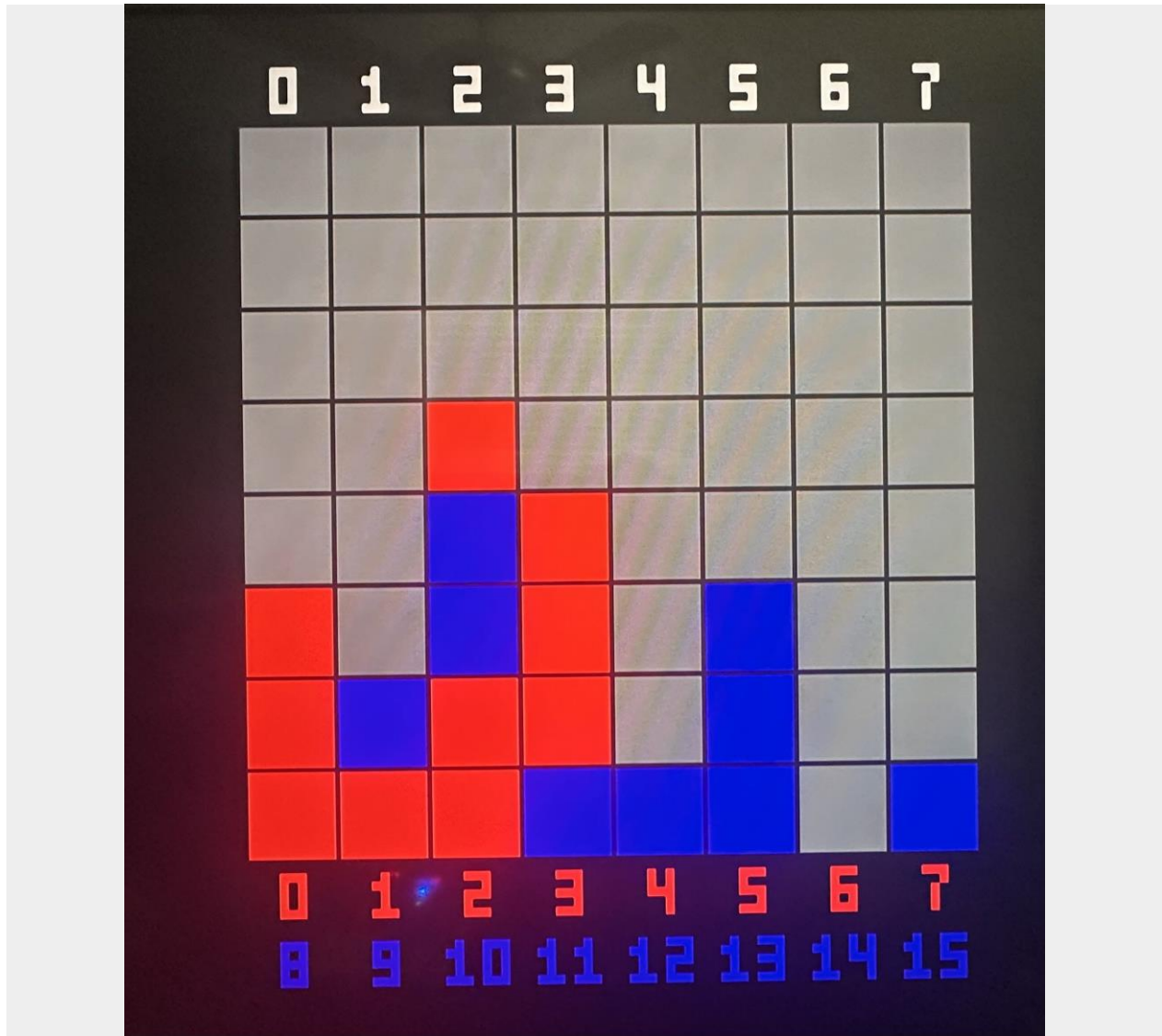
VGA

The board's VGA connector outputs 640x480 at 60 Hz. The screen layout from top to bottom is:

- Top: a 30 pixel safe area for TV overscan
- A 30 pixel row of white column numbers 0 through 7
- The 8x8 game board in the middle, with 40x40 pixel cells and 1 pixel grid lines between them
- A 30 pixel row of red digits 0 through 7, labeling P1's switch numbers
- A 30 pixel row of blue digits 8 through 15, labeling P2's switch numbers
- A 40 pixel safe area for overscan at the bottom

Pieces show as red for Player 1 and blue for Player 2. Empty cells are medium gray. When a switch is currently held high, the corresponding column in the SW label row dims red or blue to confirm the selection.

Screenshot of VGA Display is on next page



3. Finite State Machine

The main FSM is in game_fsm.v. It has 7 states:

Code	State Name	Description
0	STATE_PLAY_P1	Player 1's turn, waiting for input
1	STATE_PLAY_P2	Player 2's turn, waiting for input
2	STATE_CHECK	Evaluate the board after a move
3	STATE_WIN_P1	Player 1 has won, terminal state
4	STATE_WIN_P2	Player 2 has won, terminal state
5	STATE_DRAW	Board full, no winner, terminal state

Code	State Name	Description
6	STATE_COMMIT	1 cycle settle between PLAY and CHECK

The normal flow is: PLAY_P1, COMMIT, CHECK, PLAY_P2, COMMIT, CHECK, PLAY_P1, and so on. When a player flips a switch in their PLAY state, the FSM writes the new piece to the game board, pushes the move to the log, and transitions to STATE_COMMIT.

STATE_COMMIT exists because of a Verilog timing issue we discovered while running the testbench. Non blocking assignments take effect at the next clock edge, so the cells of the game board do not actually update until one cycle after we ask them to. If we went straight from PLAY to CHECK, the win detector would see the old board without the new piece in it, and it would miss wins. STATE_COMMIT gives the board a cycle to settle before CHECK runs. We also moved the UART per move emission trigger into COMMIT for the same reason.

In CHECK, the FSM looks at four signals: p1_win and p2_win from the win detector, board full from the game board, and last player which is registered from whichever PLAY state we came from. Based on those, the FSM goes to WIN_P1, WIN_P2, DRAW, or the other player's PLAY state.

The three win and draw states are terminal. The only way out is BTNC reset, which resets the whole FSM back to PLAY_P1.

There are two smaller FSMs in the UART path. uart_emitter has a 3 state FSM (IDLE, BYTE, NEXT) that converts move data into ASCII strings and writes them to UART FIFO. uart_tx has a 4 state FSM (IDLE, START, DATA, STOP) that handles the actual serial transmission, generating start bits, shifting out 8 data bits, then finishing with a stop bit. The baud counter inside uart_tx counts clock cycles to hit the right 9600 baud timing.

4. FIFO

We need a FIFO between the byte emitter and the UART transmitter because they operate at very different speeds. The emitter can push bytes at the 100 MHz system clock rate, which is basically instant. The UART transmitter can only send one byte every 10416 clock cycles (at 9600 baud each bit takes about 10416 cycles, and each byte is 10 bits when you count the start and stop bits). Without a FIFO in between, the emitter would have to stall on every byte, and the manual dump function (which sends the whole game history at once) would not work cleanly.

Our FIFO is in fifo.v. It is a generic synchronous FIFO with parameters for data width and depth. In our project we instantiate it with width 8 (for ASCII bytes) and depth 32. Inside the FIFO there is a small RAM (32 entries of 8 bits each, 256 bits total) plus a read pointer, a write pointer, and a count register that tracks how many entries are in use.

The FIFO accepts writes when not full and serves reads when not empty. The empty and full flags come from comparing the count register against 0 and the depth. If both a read and a write happen on the same cycle, the count stays the same and both pointers advance by 1.

The FIFO is wrapped inside uart_tx.v. From outside, uart_tx looks like a black box that accepts bytes one at a time through a write enable and a data input, with a full flag for backpressure. Nothing outside needs to know that there is a FIFO inside.

5. Modeling Styles

The project uses all four modeling styles from the project requirements.

Gate-level modeling

primitives_mux.v implements a 4 bit 2 to 1 multiplexer using only gate primitives (not, and, or). For each output bit it computes:

- sel_n = NOT sel
- a_gated = a AND sel_n
- b_gated = b AND sel
- y = a_gated OR b_gated

This is gate level because it directly instantiates logic gates with no behavioral or dataflow constructs.

Structural modeling

connect8_top.v is entirely structural. It declares wires and instantiates submodules with named port connections, with no internal logic. led_driver.v is partly structural because it instantiates 4 copies of primitives_mux to build a 16 bit wide multiplexer. uart_tx.v is also structural because it instantiates fifo as a submodule.

Behavioral modeling

game_fsm.v is the main example. It uses always @(posedge clk) blocks with case statements, if-else, and non blocking assignments to describe what the design does without specifying gates. clk_divider.v and debouncer.v also use behavioral style.

Dataflow modeling

fifo.v uses dataflow style with continuous assign statements for the empty, full, and dout signals. The win_detector.v output is computed using dataflow with concatenations and reductions inside a generate block. led_driver.v mixes structural (the mux instantiation) with dataflow (the RGB output combinational logic).

6. Testbench Strategy

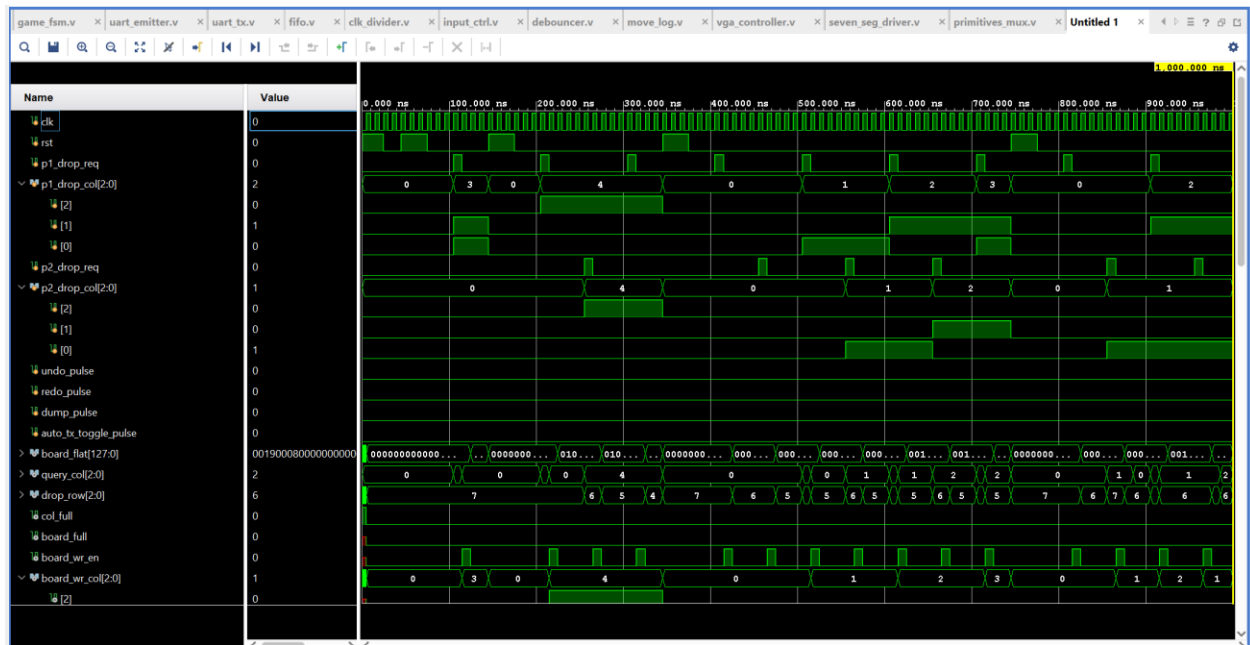
The master testbench is in connect8_tb.v. We chose to instantiate the core game pipeline (game_board, move_log, win_detector, game_fsm, uart_emitter, uart_tx) directly in the testbench instead of going through connect8_top. The input debouncers and the output display drivers (VGA, 7 segment, LEDs) are not instantiated because they either need slow real time inputs that take a long time to simulate, or produce visual outputs that are easier to verify on hardware than in simulation.

We made 10 reusable tasks to keep the test code clean and readable:

- reset_dut: asserts reset, clears all inputs, then releases reset
- play_move(player, col): handles all the timing of a drop including COMMIT and CHECK
- pulse_undo, pulse_redo, pulse_dump: each press the corresponding button for one cycle
- wait_uart_idle: waits for the emitter to finish all queued bytes
- expect_state, expect_cell, expect_eq, expect_uart_byte: check actual values against expected ones and increment pass / fail counters

A separate always block captures every byte that the emitter writes to the UART. This lets us verify the UART output without simulating the actual serial protocol bit by bit.

The testbench has 12 named test cases:



```

COM4 - PuTTY
P1:0
P2:5
P1:2
P2:3
P1:2
P2:2
P1:3
P2:2
P1:0
P2:7
P1:1
P2:4
P1:3
P2:1
P1:3
P2:5
P1:0
P2:5
P1:2
P2:3
P1:2
E::

```

LINK TO DEMO VIDEO: <https://youtu.be/usMYjRyAQFg>

8. Conclusion

The project meets all the requirements from the project specification. All six required I/O types are used in real ways that are part of the game and not just decoration. The design uses all four modeling styles (gate level, structural, behavioral, dataflow), as well as the required language features and digital design

concepts (FIFO, FSM, for-generate, parameters, defines, localparam, master testbench, tasks, for loops in the testbench).

The biggest challenge was a timing bug in the FSM that we caught while writing the testbench. The win detector was reading the board one cycle too early, before the new piece had been written. We added a STATE_COMMIT settle state between PLAY and CHECK to give the board a cycle to update. Without the testbench we probably would not have noticed this until we tried to demo a winning move on hardware and the FSM did not catch it.

Our final design passes all 65 testbench assertions and works on the actual Nexys A7 board. The 7 segment display shows the right player and move count, the LEDs animate correctly on wins, the VGA output is clean with proper grid lines and column labels, and PuTTY receives the expected move strings over UART.